

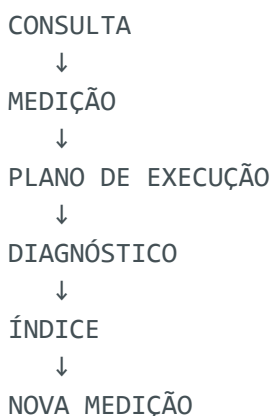
Roteiro de Laboratório — Performance Tuning no SQL Server

1. Objetivo

Demonstrar, em uma tabela de grande volume de dados fiscais, como o SQL Server executa consultas sem índices e como diferentes estratégias de indexação podem alterar:

- tempo de execução;
- consumo de CPU;
- `logical reads`;
- `physical reads`;
- quantidade de linhas processadas;
- operadores do plano de execução;
- `Table Scan`;
- `Index Scan`;
- `Index Seek`;
- `RID Lookup`;
- `Key Lookup`;
- `Sort`;
- `Hash Match`;
- custo de manutenção dos índices.

A metodologia será:



2. Banco e tabela utilizada

```
USE [933000081200000173202662_20260127_104141];  
GO
```

Tabela:

```
fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
```

Essa tabela representa itens de notas fiscais. Uma única NF-e pode possuir diversos itens:

```
1 NF-e  
  |  
  | item 1  
  | item 2  
  | item 3  
  | ...  
  | item N
```

Portanto, a tabela de itens tende a possuir cardinalidade muito maior que uma tabela de cabeçalho de NF-e.

Característica importante

O DDL apresentado não possui:

- **PRIMARY KEY;**
- **UNIQUE;**
- **CLUSTERED INDEX;**
- **NONCLUSTERED INDEX.**

O cenário inicial será, portanto, tratado como uma tabela **heap**.

3. Etapa 1 — Conhecer o volume da tabela

Comece medindo a quantidade de registros:

```
SELECT
    COUNT_BIG(*) AS quantidade_registros
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

Depois:

```
EXEC sp_spaceused
    'fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01';
```

Registre os resultados

Informação	Resultado
Quantidade de registros	
Espaço reservado	
Espaço de dados	
Espaço de índices	

Objetivo

Demonstrar que o custo de uma consulta depende também do volume de dados que precisa ser processado.

4. Etapa 2 — Verificar os índices existentes

Execute:

```
EXEC sp_helpindex  
    'fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01';
```

Registre os índices existentes.

Se o resultado confirmar que não há índices, o cenário será:

```
HEAP  
|  
▼  
Table Scan
```

Questão para discussão

Por que o SQL Server precisa percorrer a tabela quando não existe uma estrutura de acesso adequada para a consulta?

5. Etapa 3 — Criar a consulta pesada

A primeira consulta fará uma análise fiscal agrupando os dados por:

- ano;
- mês;
- UF do emitente;
- CFOP.

Além disso, serão calculados diversos indicadores.

```
SELECT  
    NF_ANO,  
    NF_MES,  
    NF_UF_EMITENTE,  
    ITEMNF_CFOP,  
  
    COUNT_BIG(*) AS quantidade_itens,  
  
    COUNT(DISTINCT NF_CHAVE_ACESSO) AS quantidade_notas,  
  
    SUM(ITEMNF_VPROD) AS valor_produtos,  
  
    SUM(ITEMNF_VDESC) AS valor_desconto,
```

```

SUM(ITEMNF_VFRETE) AS valor_frete,

SUM(ITEMNF_VSEG) AS valor_seguro,

SUM(ITEMNF_VOUTRO) AS outras_despesas,

SUM(ITEMNF_VBC) AS base_icms,

SUM(ITEMNF_VLICMS) AS valor_icms,

SUM(ITEMNF_VBCST) AS base_icms_st,

SUM(ITEMNF_VICMSST) AS valor_icms_st,

SUM(ITEMNF_VBC_IPI) AS base_ipi,

SUM(ITEMNF_VIPI) AS valor_ipi,

AVG(ITEMNF_VPROD) AS valor_medio_item,

MIN(ITEMNF_VPROD) AS menor_item,

MAX(ITEMNF_VPROD) AS maior_item

FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01

GROUP BY
    NF_ANO,
    NF_MES,
    NF_UF_EMITENTE,
    ITEMNF_CFOP

ORDER BY
    NF_ANO,
    NF_MES,
    NF_UF_EMITENTE,
    ITEMNF_CFOP;

```

A consulta combina:

```

COUNT_BIG
COUNT(DISTINCT)
SUM
AVG
MIN
MAX
GROUP BY
ORDER BY

```

sobre uma tabela potencialmente muito grande.

6. Etapa 4 — Medir o cenário inicial

Ative as estatísticas:

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;
```

Execute a consulta.

Depois desative:

```
SET STATISTICS IO OFF;  
SET STATISTICS TIME OFF;
```

Registre:

Métrica	Sem índice
CPU time	
Elapsed time	
Logical reads	
Physical reads	
Operador principal	
Memória	

7. Etapa 5 — Capturar o plano de execução

No SQL Server Management Studio:

```
Ctrl + M
```

Execute novamente a consulta.

Como a tabela é um heap, é esperado encontrar algum operador semelhante a:

```
Table Scan
```

Uma representação conceitual:

```
Table Scan
  ↓
Compute Scalar
  ↓
Hash Match / Aggregate
  ↓
Sort
  ↓
SELECT
```

Questão principal

Por que o SQL Server precisou ler tantas linhas?

O objetivo não é apenas identificar o operador mais caro, mas compreender **por que o otimizador escolheu aquele plano**.

8. Etapa 6 — Criar o primeiro índice

Crie um índice sobre a data de emissão:

```
CREATE INDEX IX_TB202_NF_DHEMI
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_DHEMI
);
```

Verifique:

```
EXEC sp_helpindex
    'fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01';
```

9. Etapa 7 — Criar uma consulta seletiva

Agora vamos analisar apenas um período.

Exemplo: janeiro de 2026.

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,

    COUNT(DISTINCT NF_CHAVE_ACESSO) AS quantidade_notas,

    SUM(ITEMNF_VPROD) AS valor_produtos,

    SUM(ITEMNF_VDESC) AS valor_desconto,

    SUM(ITEMNF_VLICMS) AS valor_icms,

    SUM(ITEMNF_VICMSST) AS valor_icms_st,

    SUM(ITEMNF_VIPI) AS valor_ipi

FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01

WHERE NF_DHEMI >= '20260101'
      AND NF_DHEMI < '20260201';
```

Agora o SQL Server pode utilizar o índice para procurar diretamente o intervalo solicitado.

Conceitualmente:

```
IX_TB202_NF_DHEMI
  |
  ▼
Index Seek
  |
  ▼
somente janeiro/2026
```

10. Etapa 8 — Comparar Table Scan × Index Seek

Sem índice

```
graph TD;
    A[HEAP] --> B[Table Scan];
    B --> C[toda a tabela];
    C --> D[filtro];
```

Com índice

```
graph TD;
    A[IX_TB202_NF_DHEMI] --> B[Index Seek];
    B --> C[somente registros do período];
```

Questão para discussão

Se a consulta precisa de apenas uma pequena parcela da tabela, qual estratégia tende a ser mais eficiente?

11. Etapa 9 — Demonstrar SARGability

Agora compare duas formas de escrever a mesma consulta.

Consulta A — usando funções sobre a coluna

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,
    SUM(ITEMNF_VPROD) AS valor_produtos
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
WHERE YEAR(NF_DHEMI) = 2026
      AND MONTH(NF_DHEMI) = 1;
```

Consulta B — utilizando intervalo

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,
    SUM(ITEMNF_VPROD) AS valor_produtos
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
WHERE NF_DHEMI >= '20260101'
      AND NF_DHEMI < '20260201';
```

Execute ambas com:

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

Depois compare:

- plano de execução;
- **logical reads**;
- CPU;
- tempo total;
- operador de acesso.

Conceito

A primeira consulta aplica funções sobre a coluna:

```
YEAR(NF_DHEMI)
MONTH(NF_DHEMI)
```

A segunda trabalha diretamente com a coluna:

```
NF_DHEMI >= ...  
AND NF_DHEMI < ...
```

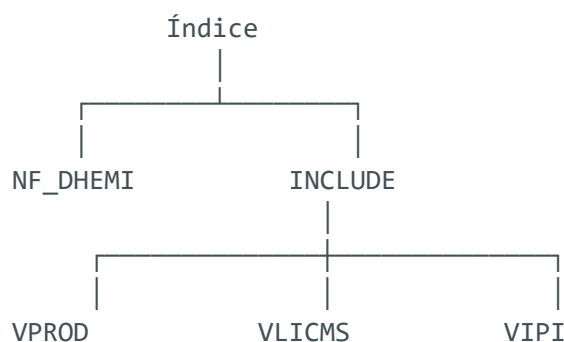
A segunda forma é mais favorável à utilização eficiente de índices e é um exemplo clássico de uma condição **SARGable**.

12. Etapa 10 — Criar um índice de cobertura

Agora vamos criar um índice que contenha a data como chave e as principais colunas utilizadas pela consulta como **INCLUDE**.

```
CREATE INDEX IX_TB202_NF_DHEMI_COVERING  
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01  
(  
    NF_DHEMI  
)  
INCLUDE  
(  
    NF_CHAVE_ACESSO,  
    ITEMNF_VPROD,  
    ITEMNF_VDESC,  
    ITEMNF_VLICMS,  
    ITEMNF_VICMSST,  
    ITEMNF_VIPI  
);
```

Conceitualmente:



O objetivo é reduzir a necessidade de buscar dados adicionais na estrutura principal.

13. Etapa 11 — Procurar RID Lookup

Observe o plano de execução.

Como a tabela original é um heap, pode aparecer:

RID Lookup

Uma representação:

```
graph TD
    A[Index Seek] --> B[encontra os registros]
    B --> C[RID Lookup]
    C --> D[Heap]
```

O índice de cobertura pode reduzir ou eliminar essa necessidade para as colunas incluídas.

Questão

Qual é o custo de utilizar um índice que encontra a linha, mas não possui todas as informações necessárias para produzir o resultado?

14. Etapa 12 — Criar um índice composto

Agora vamos testar um índice alinhado às dimensões da consulta analítica:

```
CREATE INDEX IX_TB202_PERIODO_UF_CFOP
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_ANO,
    NF_MES,
```

```
NF_UF_EMITENTE,  
ITEMNF_CFOP  
);
```

O índice está alinhado com:

```
GROUP BY  
  NF_ANO,  
  NF_MES,  
  NF_UF_EMITENTE,  
  ITEMNF_CFOP
```

Discussão

Um índice deve ser projetado considerando os **padrões reais de acesso aos dados**, e não apenas porque determinada coluna existe na tabela.

15. Etapa 13 — Comparar as estratégias

Ao final, teremos quatro cenários experimentais.

Cenário 1 — Heap

```
Table Scan
```

Cenário 2 — Índice em data

```
IX_TB202_NF_DHEMI
```

```
Index Seek / Index Scan
```

Cenário 3 — Índice de cobertura

IX_TB202_NF_DHEMI_COVERING

Index Seek

+

menor necessidade de Lookup

Cenário 4 — Índice composto

IX_TB202_PERIODO_UF_CFOP

16. Etapa 14 — Montar a tabela de resultados

Preencha durante o laboratório:

Cenário	Índice	Operador	Logical Reads	CPU	Tempo
1	Nenhum	Table Scan			
2	NF_DHEMI				
3	Covering				
4	Composto				

Análise

O aluno deverá explicar:

1. Qual cenário apresentou maior quantidade de **logical reads**?
2. Qual apresentou menor tempo?
3. Qual apresentou menor consumo de CPU?
4. O plano mudou?
5. O SQL Server utilizou **Index Seek**?
6. Houve **RID Lookup**?
7. O índice de cobertura trouxe benefício?

8. O índice composto foi utilizado?
9. O resultado da consulta permaneceu igual?

17. Etapa 15 — Demonstrar o custo dos índices

Índices não são gratuitos.

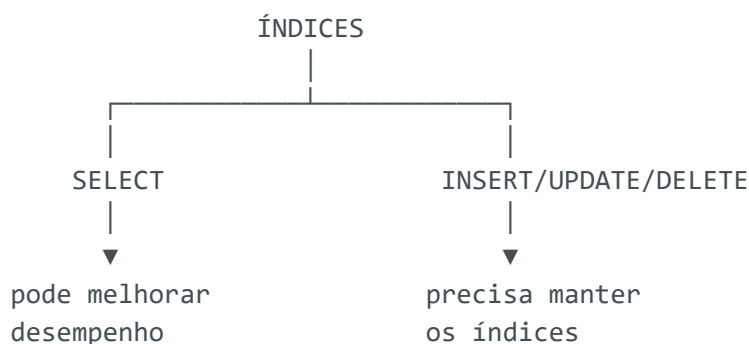
Eles podem melhorar:

SELECT

mas também introduzem custos em:

INSERT
UPDATE
DELETE

Conceitualmente:



Os custos incluem:

- espaço em disco;
- memória;
- CPU;
- manutenção;
- **INSERT**;
- **UPDATE**;
- **DELETE**;

- REBUILD;
- REORGANIZE.

18. Etapa 16 — Atualizar estatísticas

Depois das alterações:

```
UPDATE STATISTICS  
fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

Execute novamente as consultas.

Explique que o otimizador utiliza estatísticas para estimar:

quantas linhas serão retornadas?

Essa estimativa influencia decisões como:

```
Index Seek  
Index Scan  
Table Scan  
Nested Loops  
Hash Match  
Merge Join
```

19. Etapa 17 — Comparar Estimated × Actual Rows

No plano de execução, observe:

```
Estimated Number of Rows  
VS  
Actual Number of Rows
```

Exemplo hipotético:

Estimated: 10.000
Actual: 2.500.000

Uma diferença grande entre esses valores pode levar o otimizador a escolher um plano inadequado.

Questão

O que pode acontecer quando o SQL Server estima uma quantidade de linhas muito diferente da quantidade realmente processada?

20. Etapa 18 — Remover os índices experimentais

Ao finalizar o laboratório, remova os índices criados experimentalmente:

```
DROP INDEX IX_TB202_NF_DHEMI  
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

```
DROP INDEX IX_TB202_NF_DHEMI_COVERING  
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

```
DROP INDEX IX_TB202_PERIODO_UF_CFOP  
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

Atenção: execute os comandos **DROP INDEX** somente se esses índices tiverem sido criados especificamente para o laboratório. Não remova índices existentes no ambiente de produção.

21. Sequência didática recomendada

01. Conhecer a tabela
↓
02. Medir quantidade de registros
↓
03. Verificar índices existentes
↓
04. Executar consulta pesada
↓
05. STATISTICS IO / TIME
↓
06. Plano de execução
↓
07. Identificar Table Scan
↓
08. Criar índice em NF_DHEMI
↓
09. Executar consulta seletiva
↓
10. Comparar Scan × Seek
↓
11. Demonstrar SARGability
↓
12. Criar índice de cobertura
↓
13. Analisar RID Lookup
↓
14. Criar índice composto
↓
15. Comparar estratégias
↓
16. Analisar estatísticas
↓
17. Comparar Estimated × Actual Rows
↓
18. Discutir custo dos índices
↓
19. Remover índices experimentais
↓
20. Conclusão

22. Conceitos que o aluno deverá dominar

Ao final do laboratório, o aluno deverá compreender:

- Heap;
- Table Scan;
- Clustered Index;

- Nonclustered Index;
- Index Scan;
- Index Seek;
- RID Lookup;
- Key Lookup;
- índice de cobertura;
- índice composto;
- **INCLUDE**;
- SARGability;
- **STATISTICS IO**;
- **STATISTICS TIME**;
- plano de execução;
- cardinalidade;
- estimativa de linhas;
- estatísticas;
- custo de manutenção dos índices.

23. Conclusão esperada

O objetivo principal do laboratório não é demonstrar que:

"Índice deixa a consulta rápida".

O objetivo é demonstrar o processo correto de **Performance Tuning**:

```
graph TD;
  MEDIR --> OBSERVAR;
  OBSERVAR --> FORMULAR_HIPOTESE[FORMULAR HIPÓTESE];
  FORMULAR_HIPOTESE --> ALTERAR;
  ALTERAR --> MEDIR_NOVAMENTE[MEDIR NOVAMENTE];
  MEDIR_NOVAMENTE --> COMPARAR;
  COMPARAR --> VALIDAR;
```

O SQL Server pode ou não utilizar o índice criado. Portanto, não devemos assumir antecipadamente que um índice produzirá **Index Seek** ou que será necessariamente

mais rápido.

A decisão do otimizador depende de fatores como:

- cardinalidade;
- seletividade;
- distribuição dos dados;
- estatísticas;
- custo estimado;
- quantidade de páginas;
- memória disponível;
- predicados da consulta;
- índices disponíveis.

A evidência deve vir do plano de execução e das métricas de desempenho.